

Algorithmen und Datenstrukturen



Bubble-Sort

Sortierv Verfahren

Bubble-Sort

BUBBLE-SORT(Z)

```
1:  $n \leftarrow \text{length}(Z)$ 
2: for  $i \leftarrow 0$  to  $n - 2$  do
3:   for  $j \leftarrow n - 1$  downto  $i + 1$  do
4:     if  $Z[j - 1] > Z[j]$  then
5:        $\text{temp} \leftarrow Z[j]$ 
6:        $Z[j] \leftarrow Z[j - 1]$ 
7:        $Z[j - 1] \leftarrow \text{temp}$ 
8:     end if
9:   end for
10: end for
```

Lösungsidee

Das Feld wird von hinten nach vorne durchlaufen und die nebeneinander liegenden Speicherzellen verglichen. Stimmt die Reihenfolge nicht, werden die Inhalte vertauscht.

Mit dem ersten Durchgang wandert das kleinste Element an die erste Stelle. Der zweite Durchgang muss dann nur noch das Feld ab der zweiten Stelle bearbeiten.

Sortiervverfahren

Bubble-Sort

BUBBLE-SORT(Z)

```
1:  $n \leftarrow \text{length}(Z)$ 
2: for  $i \leftarrow 0$  to  $n - 2$  do
3:   for  $j \leftarrow n - 1$  downto  $i + 1$  do
4:     if  $Z[j - 1] > Z[j]$  then
5:        $\text{temp} \leftarrow Z[j]$ 
6:        $Z[j] \leftarrow Z[j - 1]$ 
7:        $Z[j - 1] \leftarrow \text{temp}$ 
8:     end if
9:   end for
10: end for
```

Vor Sortierung

85 72 48 6 22 10 37 76

Schritt 1: 6 85 72 48 10 22 37 76

Sortiervverfahren

Bubble-Sort

BUBBLE-SORT(Z)

```

1:  $n \leftarrow \text{length}(Z)$ 
2: for  $i \leftarrow 0$  to  $n - 2$  do
3:   for  $j \leftarrow n - 1$  downto  $i + 1$  do
4:     if  $Z[j - 1] > Z[j]$  then
5:        $\text{temp} \leftarrow Z[j]$ 
6:        $Z[j] \leftarrow Z[j - 1]$ 
7:        $Z[j - 1] \leftarrow \text{temp}$ 
8:     end if
9:   end for
10: end for

```

Vor Sortierung

85 72 48 6 22 10 37 76

Schritt 1:	6	85	72	48	10	22	37	76
Schritt 2:	6	10	85	72	48	22	37	76
Schritt 3:	6	10	22	85	72	48	37	76
Schritt 4:	6	10	22	37	85	72	48	76
Schritt 5:	6	10	22	37	48	85	72	76
Schritt 6:	6	10	22	37	48	72	85	76
Schritt 7:	6	10	22	37	48	72	76	85

Sortierv Verfahren

Bubble-Sort

BUBBLE-SORT(Z)

```
1:  $n \leftarrow \text{length}(Z)$ 
2: for  $i \leftarrow 0$  to  $n - 2$  do
3:   for  $j \leftarrow n - 1$  downto  $i + 1$  do
4:     if  $Z[j - 1] > Z[j]$  then
5:        $\text{temp} \leftarrow Z[j]$ 
6:        $Z[j] \leftarrow Z[j - 1]$ 
7:        $Z[j - 1] \leftarrow \text{temp}$ 
8:     end if
9:   end for
10: end for
```

Effizienzüberlegung

Speicherkomplexität

Der Sortieralgorithmus benötigt über das gegebene Feld hinaus nur temporär einzelne Speicherzellen unabhängig von der Größe von n : $O(1)$

Laufzeitkomplexität

Aufwändig sind der Vergleich (Zeile 4) und die Zuweisungen (Zeilen 5-6). Beides findet innerhalb zweier verschachtelter Schleifen statt. Die Anzahl der Vergleiche wächst mit $O(n^2)$. Diese Laufzeit ist unabhängig von der Vorbelegung des Feldes Z .

Sortiervverfahren

Bubble-Sort (Verbesserung)

BUBBLE-SORT2(Z)

```
1:  $n \leftarrow \text{length}(Z)$ 
2: repeat
3:    $\text{sortiert} \leftarrow \text{true}$ 
4:   for  $i \leftarrow 0$  to  $n - 2$  do
5:     if  $Z[i] > Z[i + 1]$  then
6:        $\text{sortiert} \leftarrow \text{false}$ 
7:        $\text{temp} \leftarrow Z[i]$ 
8:        $Z[i] \leftarrow Z[i + 1]$ 
9:        $Z[i + 1] \leftarrow \text{temp}$ 
10:    end if
11:  end for
12:   $n \leftarrow n - 1$ 
13: until  $\text{sortiert} = \text{true}$  or  $n \leq 1$ 
```

Verbesserte Lösungsidee

Das Feld wird von vorne nach hinten durchlaufen und die nebeneinander liegenden Speicherzellen verglichen. Stimmt die Reihenfolge nicht, werden die Inhalte vertauscht.

Diese Durchläufe werden wiederholt, bis im letzten Durchlauf keine Vertauschung mehr notwendig war.

Sortierv Verfahren

Bubble-Sort (Verbesserung)

BUBBLE-SORT2(Z)

```
1:  $n \leftarrow \text{length}(Z)$ 
2: repeat
3:    $\text{sortiert} \leftarrow \text{true}$ 
4:   for  $i \leftarrow 0$  to  $n - 2$  do
5:     if  $Z[i] > Z[i + 1]$  then
6:        $\text{sortiert} \leftarrow \text{false}$ 
7:        $\text{temp} \leftarrow Z[i]$ 
8:        $Z[i] \leftarrow Z[i + 1]$ 
9:        $Z[i + 1] \leftarrow \text{temp}$ 
10:    end if
11:  end for
12:   $n \leftarrow n - 1$ 
13: until  $\text{sortiert} = \text{true}$  or  $n \leq 1$ 
```

Effizienzüberlegung

Speicherkomplexität

Der Sortieralgorithmus benötigt über das gegebene Feld hinaus nur temporär einzelne Speicherzellen unabhängig von der Größe von n : $O(1)$

Laufzeitkomplexität

Im Sonderfall, dass das Feld Z bereits sortiert ist, bricht der Algorithmus nach nur einem Durchlauf ab: $O(n)$
In allen anderen Fällen bleibt es jedoch bei $O(n^2)$.

Select-Sort

Sortiervverfahren

Select-Sort

SELECT-SORT(Z)

```
1:  $n \leftarrow \text{length}(Z)$ 
2: for  $i \leftarrow 0$  to  $n - 1$  do
3:    $\text{min} \leftarrow i$ 
4:   for  $j \leftarrow i + 1$  to  $n - 1$  do
5:     if  $Z[j] < Z[\text{min}]$  then
6:        $\text{min} \leftarrow j$ 
7:     end if
8:   end for
9:    $\text{temp} \leftarrow Z[i]$ 
10:   $Z[i] \leftarrow Z[\text{min}]$ 
11:   $Z[\text{min}] \leftarrow \text{temp}$ 
12: end for
```

Lösungsidee

Zunächst wird kleinste Element ermittelt und an 1. Stelle gesetzt. Dann wird das zweitkleinste Element und an 2. Stelle gesetzt usw.

Für jede Position i wird das Feld ab der Position $i + 1$ durchsucht und ggf. das gefundene kleinste Element an die Position i getauscht.

Sortiervverfahren

Select-Sort

SELECT-SORT(Z)

```

1:  $n \leftarrow \text{length}(Z)$ 
2: for  $i \leftarrow 0$  to  $n - 1$  do
3:    $\text{min} \leftarrow i$ 
4:   for  $j \leftarrow i + 1$  to  $n - 1$  do
5:     if  $Z[j] < Z[\text{min}]$  then
6:        $\text{min} \leftarrow j$ 
7:     end if
8:   end for
9:    $\text{temp} \leftarrow Z[i]$ 
10:   $Z[i] \leftarrow Z[\text{min}]$ 
11:   $Z[\text{min}] \leftarrow \text{temp}$ 
12: end for

```

Vor Sortierung

44 81 90 63 35 43 38 38

Schritt 1:	35	81	90	63	44	43	38	38
Schritt 2:	35	38	90	63	44	43	81	38
Schritt 3:	35	38	38	63	44	43	81	90
Schritt 4:	35	38	38	43	44	63	81	90
Schritt 5:	35	38	38	43	44	63	81	90
Schritt 6:	35	38	38	43	44	63	81	90
Schritt 7:	35	38	38	43	44	63	81	90
Schritt 8:	35	38	38	43	44	63	81	90

Sortierv Verfahren

Select-Sort

SELECT-SORT(Z)

```
1:  $n \leftarrow \text{length}(Z)$ 
2: for  $i \leftarrow 0$  to  $n - 1$  do
3:    $\text{min} \leftarrow i$ 
4:   for  $j \leftarrow i + 1$  to  $n - 1$  do
5:     if  $Z[j] < Z[\text{min}]$  then
6:        $\text{min} \leftarrow j$ 
7:     end if
8:   end for
9:    $\text{temp} \leftarrow Z[i]$ 
10:   $Z[i] \leftarrow Z[\text{min}]$ 
11:   $Z[\text{min}] \leftarrow \text{temp}$ 
12: end for
```

Effizienzüberlegung

Speicherkomplexität

Der Sortieralgorithmus benötigt über das gegebene Feld hinaus nur temporär einzelne Speicherzellen unabhängig von der Größe von n : $O(1)$

Laufzeitkomplexität

Zur ersten Bestimmung des Minimums sind $n - 1$ Vergleiche notwendig, bei der zweiten $n - 2$ Vergleiche usw. Somit gilt insgesamt:

$(n - 1) + (n - 2) + \dots + 2 + 1 = \frac{n \cdot (n - 1)}{2}$ Vergleiche – unabhängig, ob das Feld bereits vorsortiert ist $\rightarrow O(n^2)$.

Insert-Sort

Sortiervverfahren

Insert-Sort

INSERT-SORT(Z)

```
1:  $n \leftarrow \text{length}(Z)$ 
2: for  $i \leftarrow 1$  to  $n - 1$  do
3:    $elem \leftarrow Z[i]$ 
4:    $j \leftarrow i$ 
5:   while  $j > 0$  and  $Z[j - 1] > elem$  do
6:      $Z[j] \leftarrow Z[j - 1]$ 
7:      $j \leftarrow j - 1$ 
8:   end while
9:    $Z[j] \leftarrow elem$ 
10: end for
```

Lösungsidee

Man geht wie ein Schafkopf-Spieler vor, der Karten einzeln aufnimmt und jede in das bereits auf der Hand befindliche Blatt an der richtigen Stelle einfügt.

Am Anfang ist (trivialerweise) nur die erste Position (Index 0) sortiert. Beginnend bei $i = 1$ werden alle anderen Elemente in die sortierte Liste eingefügt. Um Platz für $Z[i]$ zu schaffen, müssen ggf. die sortierten Elemente um eine Stelle nach rechts geschoben werden.

Sortiervverfahren

Insert-Sort

INSERT-SORT(Z)

```

1:  $n \leftarrow \text{length}(Z)$ 
2: for  $i \leftarrow 1$  to  $n - 1$  do
3:    $elem \leftarrow Z[i]$ 
4:    $j \leftarrow i$ 
5:   while  $j > 0$  and  $Z[j - 1] > elem$  do
6:      $Z[j] \leftarrow Z[j - 1]$ 
7:      $j \leftarrow j - 1$ 
8:   end while
9:    $Z[j] \leftarrow elem$ 
10: end for

```

Vor Sortierung

44 39 29 58 27 65 1 79 46 10 62 81

Schritt 1:	39	44	29	58	27	65	1	79	46	10	62	81
Schritt 2:	29	39	44	58	27	65	1	79	46	10	62	81
Schritt 3:	29	39	44	58	27	65	1	79	46	10	62	81
Schritt 4:	27	29	39	44	58	65	1	79	46	10	62	81
Schritt 5:	27	29	39	44	58	65	1	79	46	10	62	81
Schritt 6:	1	27	29	39	44	58	65	79	46	10	62	81
Schritt 7:	1	27	29	39	44	58	65	79	46	10	62	81
Schritt 8:	1	27	29	39	44	46	58	65	79	10	62	81
Schritt 9:	1	10	27	29	39	44	46	58	65	79	62	81
Schritt 10:	1	10	27	29	39	44	46	58	62	65	79	81
Schritt 11:	1	10	27	29	39	44	46	58	62	65	79	81

Sortierv Verfahren

Insert-Sort

INSERT-SORT(Z)

```
1:  $n \leftarrow \text{length}(Z)$ 
2: for  $i \leftarrow 1$  to  $n - 1$  do
3:    $elem \leftarrow Z[i]$ 
4:    $j \leftarrow i$ 
5:   while  $j > 0$  and  $Z[j - 1] > elem$  do
6:      $Z[j] \leftarrow Z[j - 1]$ 
7:      $j \leftarrow j - 1$ 
8:   end while
9:    $Z[j] \leftarrow elem$ 
10: end for
```

Effizienzüberlegung

Speicherkomplexität

Der Sortieralgorithmus benötigt über das gegebene Feld hinaus nur temporär einzelne Speicherzellen unabhängig von der Größe von n : $O(1)$

Sortierv Verfahren

Insert-Sort

INSERT-SORT(Z)

```
1:  $n \leftarrow \text{length}(Z)$ 
2: for  $i \leftarrow 1$  to  $n - 1$  do
3:    $elem \leftarrow Z[i]$ 
4:    $j \leftarrow i$ 
5:   while  $j > 0$  and  $Z[j - 1] > elem$  do
6:      $Z[j] \leftarrow Z[j - 1]$ 
7:      $j \leftarrow j - 1$ 
8:   end while
9:    $Z[j] \leftarrow elem$ 
10: end for
```

Effizienzüberlegung

Laufzeitkomplexität

Innerhalb der for-Schleife werden im Kopf der while-Schleife Vergleiche durchgeführt.

Best Case:

Die Laufbedingung in Zeile 5 ist nie erfüllt, dann werden nur $O(n)$ Vergleiche durchgeführt. Das ist der Fall, wenn das Feld bereits sortiert ist.

Sortierv Verfahren

Insert-Sort

INSERT-SORT(Z)

```
1:  $n \leftarrow \text{length}(Z)$ 
2: for  $i \leftarrow 1$  to  $n - 1$  do
3:    $elem \leftarrow Z[i]$ 
4:    $j \leftarrow i$ 
5:   while  $j > 0$  and  $Z[j - 1] > elem$  do
6:      $Z[j] \leftarrow Z[j - 1]$ 
7:      $j \leftarrow j - 1$ 
8:   end while
9:    $Z[j] \leftarrow elem$ 
10: end for
```

Effizienzüberlegung

Laufzeitkomplexität

Innerhalb der for-Schleife werden im Kopf der while-Schleife Vergleiche durchgeführt.

Worst Case:

Jedes neu betrachtete Element muss komplett an den Anfang durchgeschoben werden (d.h. das Feld war umgekehrt sortiert. Dann sind es

$$(n - 1) + (n - 2) + \dots + 2 + 1 = \frac{n \cdot (n - 1)}{2} \text{ Vergleiche}$$

$\rightarrow O(n^2)$.

Sortierv Verfahren

Insert-Sort

INSERT-SORT(Z)

```
1:  $n \leftarrow \text{length}(Z)$ 
2: for  $i \leftarrow 1$  to  $n - 1$  do
3:    $elem \leftarrow Z[i]$ 
4:    $j \leftarrow i$ 
5:   while  $j > 0$  and  $Z[j - 1] > elem$  do
6:      $Z[j] \leftarrow Z[j - 1]$ 
7:      $j \leftarrow j - 1$ 
8:   end while
9:    $Z[j] \leftarrow elem$ 
10: end for
```

Effizienzüberlegung

Laufzeitkomplexität

Innerhalb der for-Schleife werden im Kopf der while-Schleife Vergleiche durchgeführt.

Average Case:

Im Mittel muss jedes neu betrachtete Element bis zur Mitte der sortierten Anfangssequenz geschoben werden:

$$\frac{n-1}{2} + \frac{n-2}{2} + \dots + \frac{2}{2} + \frac{1}{2} = \frac{n \cdot (n-1)}{4} \text{ Vergleiche}$$

$\rightarrow O(n^2)$.

Vielen Dank für die Aufmerksamkeit!